

BootIT

Ai Ting Lee

Based on slides by Niek Janssen

-- Based on slides by Sebastian Mateos Nicolajsen

---- Based on slides by Claus Brabrand

Welcome
Back!!!



Agenda

- **Recap**
- Conditional Statements
 - If Statements
 - If-else Statements
 - Nested if(-else) Statements
- Booleans
- Returning Functions
- Boolean Comparisons
- Iterations / Loops
- Interactive Programs

Variables

Type

Name

Value

```
int x = 42;  
System.out.println(x);
```



Value: What is actually stored?

Name: Identifier used to refer to this box.

int

Type: Shape of the box; What can be put in?

Types

int

```
int x = 42;  
System.out.println(x);
```

1, 2, 0, -2, -420, 1000000, 2147483647

String

```
String s = "hi";  
System.out.println(s);
```

"hi", "hello world", "@#\$%&*(),.,;", "14"

double

```
double d = 3.14;  
System.out.println(d);
```

1.5, 3.1415, -27.15, 1.0, 6.02E23

boolean

```
boolean b = true;  
System.out.println(b);
```

true and false

Operations on Numbers

Operators in **int**

```
System.out.println(3 + 3);
```

```
int x = 3;  
System.out.println(x + x);
```

 6

```
x = 2;  
System.out.println(x * x);
```

 4

```
x = 6;  
System.out.println(x / 3);
```

 2

Operators in **double**

```
System.out.println(2.2 + 2.2);
```

```
double x = 2.2;  
System.out.println(x + x);
```

 4.4

```
x = 2.5;  
System.out.println(x * 3);
```

 7.5

```
x = 5.0;  
System.out.println(x / 2);
```

 2.5

Operations on Strings

+ is String Concatenation Operator

```
String hi = "Hello ";  
String world = "World!";  
String greet = hi + world;  
  
System.out.println(greet);  
// Hello World!
```

It also works between Strings and numbers:

```
int year = 2026;  
System.out.println("The year is " + year);
```

Methods

Methods let us **reuse code** and give a snippet a clear **responsibility**.

This does exactly the same as the previous exercise, wrapped in a method:

```
public class Main {  
    static void dkkToEur() {  
        double dkk = 100;  
        double eur = dkk / 7.45;  
        System.out.println(dkk + " kr corresponds to " + eur + " euro");  
    }  
  
    public static void main(String[] args) {  
        dkkToEur();  
    }  
}
```


Methods with Parameters

But the previous method always converts 100 kr.

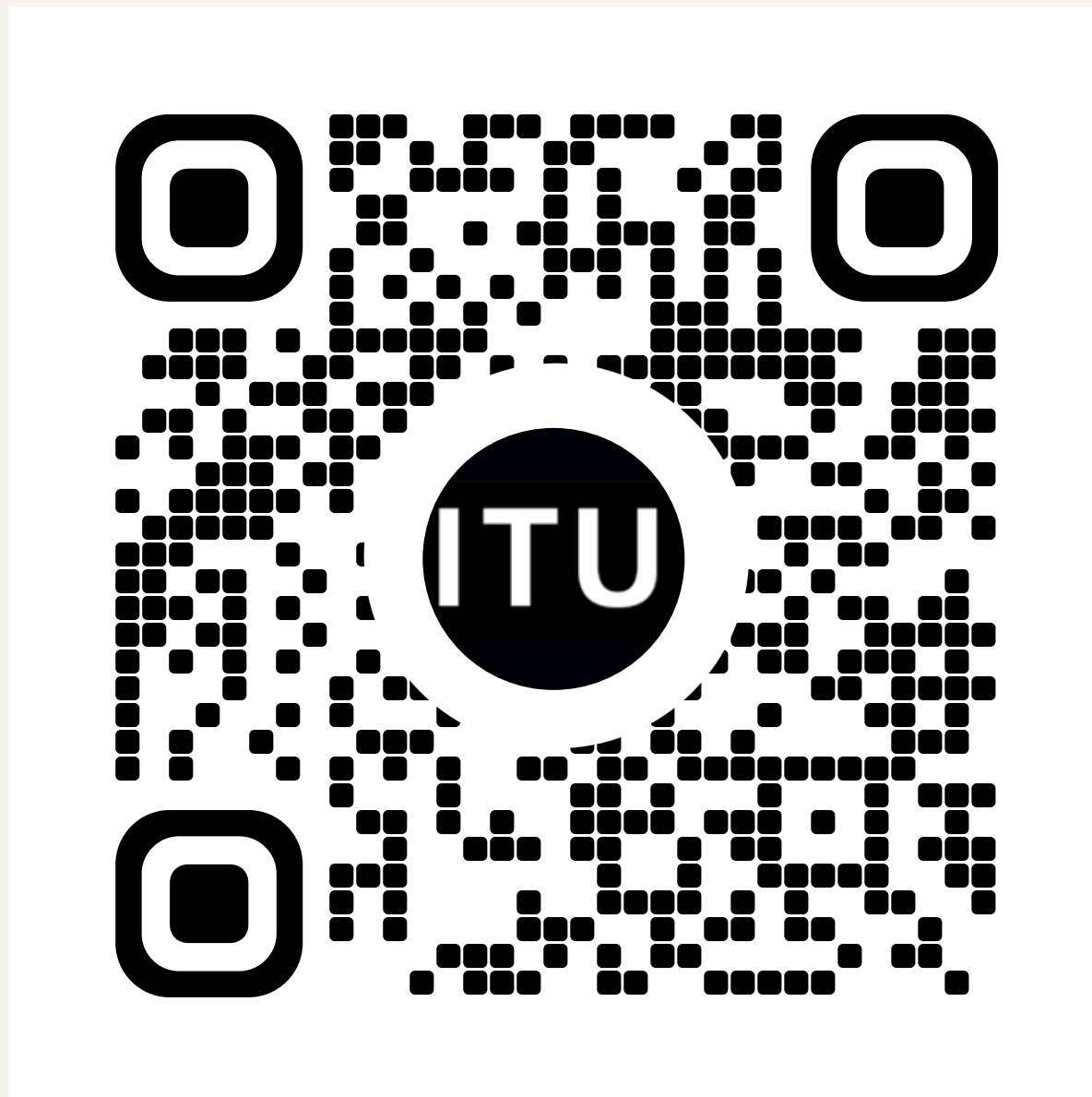
With a **parameter**, the same method works for any value:

```
public class Main {  
    static void dkkToEur(double dkk) {  
        double dkk = 100;  
        double eur = dkk / 7.45;  
        System.out.println(dkk + " kr corresponds to " + eur + " euro");  
    }  
  
    public static void main(String[] args) {  
        dkkToEur();  
        dkkToEur(100);  
        dkkToEur(20);  
    }  
}
```

BootCode - Online IDE Built for BootIT

Again! Access the website via a laptop by...

Scanning the QR code



Join today's session, refresh if you don't see it.

Entering the web address directly

BootCode - ITU BootIT

BootCode is the beginner-friendly Java coding platform for ITU BootIT, the pre-master computer science bootcamp. Write, run, and submit Java exercises directly in your browser.

cobblerscoders.tech

<https://cobblerscoders.tech/>

Clicking the link on the course platform



<https://learnit.itu.dk/course/view.php?id=3026730>



Agenda

- Recap
- **Conditional Statements**
 - If Statements
 - If-Else Statements
 - Nested if(-else) Statements
 - If-Else-If Ladder
- Booleans, Logical and Comparison Operators
- Returning Functions
- Iterations / Loops
- Interactive Programs

If Statements

- An **if** runs a block of code only when a condition is true. If the condition is false, Java simply skips the block and continues.

```
if (condition) {  
    System.out.println("Yes the condition is correct");  
}
```

Exercise - Conditional Printing

11

```
if (condition) {  
    System.out.println("Yes the condition is correct");  
}
```

You just walked into the ITU atrium. You have a boolean **atItu**.
Print "Welcome to ITU!" only if **atItu** is **true**.
If you set **atItu** to **false**, the program should print nothing.

If-Else Statements

- With **if-else**, exactly one of the two branches runs

```
// code that is always executed

if (condition) {
    System.out.println("To be");
} else {
    System.out.println("Not to be");
}

// code that is always executed
```

Exercise - Printing with If-Else

12

```
// code that is always executed

if (condition) {
    System.out.println("To be");
} else {
    System.out.println("Not to be");
}

// code that is always executed
```

Fun fact: Scrollbar is the Friday bar at ITU — open every Friday after 15:00 during the semester.

Write a method that prints whether it is Scrollbar day for a given weekday:

- static void **IsScrollBarOpen(String weekday)**
- **If weekday == "Friday"** → Yes, it is Friday, Scrollbar will open today!
- **Otherwise** → No, Scrollbar is closed.

Call it twice from main, once with weekday="Friday" and another with "Thursday"

Nested if Statements

You can put an if inside another if. The inner block only runs when the outer condition is also true.

```
int number = 42;

if (number > 0) {
    if (number > 100) {
        System.out.println("positive and big");
    } else {
        System.out.println("positive but small");
    }
} else {
    System.out.println("not positive");
}
```


Exercise - Printing with Nested Ifs

13

```
int number = 42;

if (number > 0) {
    if (number > 100) {
        System.out.println("positive and big");
    } else {
        System.out.println("positive but small");
    }
} else {
    System.out.println("not positive");
}
```

The ITU canteen serves lunch from Monday to Friday, 11:00 to 14:00.

Declare **isCanteenOpen(boolean isWeekday, double hour)** so it prints whether the canteen is open, using **nested if**, three layers deep:

1. Outer layer — is it a weekday? Use the parameter **isWeekday**.
2. Middle layer (only checked when it is a weekday) — is **hour < 11.0**?
3. Inner layer (only checked when the middle layer is false) — is hour **>= 14.0**?

If-Else-If Ladder

You can chain multiple conditions using **else if**. In this chain, only the first (from top to bottom) matching condition will execute.

```
if (number >= 100) {  
    System.out.println("3+ digits");  
} else if (number >= 10) {  
    System.out.println("2 digits");  
} else {  
    System.out.println("1 digit");  
}
```

Exercise - Printing with an If-Else-If Ladder

14

```
if (number >= 100) {  
    System.out.println("3+ digits");  
} else if (number >= 10) {  
    System.out.println("2 digits");  
} else {  
    System.out.println("1 digit");  
}
```

The ITU Canteen offers a late lunch discount on weekdays after 13:45. Assuming it is already a weekday, **implement** the lunch-hour check using a single **if-else-if ladder** that includes the late lunch discount logic and **print**:

Use a **double** for the time of day (e.g. 11.0 = 11:00, 13.75 = 13:45):

- Earlier than 11.0 (< 11.0) → Too early. Lunch starts at 11:00.
- Between 11.0 and 13.75 (≥ 11.0 and < 13.75) → Lunch is being served at full price.
- Between 13.75 and 14.0 (≥ 13.75 and < 14.0) → Lunch is being served with a late lunch discount!
- After 14.0 (≥ 14) → Too late - lunch ended at 14:00.



Agenda

- Recap
- Conditional Statements
 - If Statements
 - If-Else Statements
 - Nested if(-else) Statements
 - If-Else-If Ladder
- **Booleans, Logical and Comparison Operators**
- Returning Functions
- Iterations / Loops
- Interactive Programs

Boolean

- A Boolean variable can take only two values: **true** or **false**

```
if(true) {  
    System.out.println("This will ALWAYS be printed");  
}  
  
if(false) {  
    System.out.println("This will NEVER be printed");  
}
```

- Once you have a boolean, you can branch on it directly.

```
boolean isRaining = true;  
  
if (isRaining) {  
    System.out.println("Study");  
} else {  
    System.out.println("Enjoy the sun");  
}
```


Exercise - Branching with Booleans

15

```
boolean isRaining = true;

if (isRaining) {
    System.out.println("Study");
} else {
    System.out.println("Enjoy the sun");
}
```

The ITU fitness room (located in the basement, beneath Scrollbar) requires a membership.

Declare **checkFitnessAccess(boolean hasMembership)** so it stores membership in a **boolean** parameter, then **branches on the boolean** directly.

Print “Accessed” if they have a membership; **otherwise**, print “Not allowed”.

Logical Operators

- ==** 1 == 1 is equal to -> true
- !=** 1 != 2 is not equal to -> true
- <** 1 < 2 is less than -> true
- >** 2 > 1 is greater than -> true
- <=** 1 <= 2 is less than or equal to -> true
- >=** 2 >= 2 is greater than or equal to -> true

Comparison Operators

! `!(1 > 2)` not -> true, because (1 > 2) is false

&& `(2 > 1) && (3 > 2)` and -> true, both sides are true

|| `(2 < 1) || (2 == 2)` or -> true, at least one side is true

P	Q	!P	P && Q	P Q
T	T	F	T	T
T	F	F	F	T
F	T	T	F	T
F	F	T	F	F

Exercise - Branching with Boolean Operators

16

== 1 == 1 is equal to -> true	! !(1 > 2) not -> true, because (1 > 2) is false
!= 1 != 2 is not equal to -> true	&& (2 > 1) && (3 > 2) and -> true, both sides are true
< 1 < 2 is less than -> true	 (2 < 1) (2 == 2) or -> true, at least one side is true
> 2 > 1 is greater than -> true	
<= 1 <= 2 is less than or equal to -> true	
>= 2 >= 2 is greater than or equal to -> true	

The ITU fitness room offers a free trial on the first Tuesday of every month.

Now, you need to update the ITU fitness room access logic.

Declare **checkFitnessAccess(boolean hasMembership, boolean isFreeTrialTuesday)**:
a student may enter if they have a membership or if it is free-trial Tuesday.

Print "Accessed"; otherwise, print "Not allowed".



Agenda

- Recap
- Conditional Statements
 - If Statements
 - If-Else Statements
 - Nested if(-else) Statements
 - If-Else-If Ladder
- Booleans, Logical and Comparison Operators
- **Returning Functions**
- Iterations / Loops
- Interactive Programs

Returning Values from Methods

- Instead of **void** (no return values), a method can send a specific value back to the caller using the **return** keyword.
- The **data type** being returned must be declared in the method **signature** exactly where void used to be.

```
public class Main {  
    public static int addNumbers(int a, int b) {  
        // Returns the calculated result to the caller  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        int result = addNumbers(2, 3);  
        System.out.println(result);  
    }  
}
```

Exercise - Implementing a Method with a Return Value

17

```
public class Main {  
    public static int addNumbers(int a, int b) {  
        // Returns the calculated result to the caller  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        int result = addNumbers(2, 3);  
        System.out.println(result);  
    }  
}
```

Glass boxes are group study rooms in the ITU building.
You can reserve them on the same day at the info desk.
Some of these rooms operate on a "first-come, first-served" basis.

Declare a method named **IsFirstComeFirstServe** that returns a boolean.
The complete list of these room numbers is: 2A03, 2A07, 3A03, 4A01, 4A03, 4A07, 5A03, 5A07.

Make sure the method returns the correct values for the examples called in the main function.



Agenda

- Recap
- Conditional Statements
 - If Statements
 - If-Else Statements
 - Nested if(-else) Statements
 - If-Else-If Ladder
- Booleans, Logical and Comparison Operators
- Returning Functions
- **Iterations / Loops**
- Interactive Programs

While Loops

- A **loop** is code that is **executed** many times

```
System.out.println("I should be careful with AI code!");  
System.out.println("I should be careful with AI code!");  
System.out.println("I should be careful with AI code!");  
System.out.println("I should be careful with AI code!");  
System.out.println("I should be careful with AI code!");  
... x10
```



```
public static void main(String[] args) {  
    int i = 0; // initialization  
    while ( i < 10 ) { // loop condition  
        System.out.println("I should be careful with AI code!");  
        i = i + 1; // incrementation  
    }  
}
```

i starts at 0

condition $i < 10$

i runs from 0 to 9 (10 times).

While Loops

- A while loop keeps running as long as its condition evaluates to true.
- The loop condition can be any **boolean expression**—including a boolean variable or a method that returns a boolean!
- The loop checks the condition **before** every iteration. Once the condition becomes false, the loop immediately stops and skips the body.
- You **must** update the variables involved in your loop condition. Otherwise you may create an **infinite loop** that runs forever and crashes your program!
- **Bad** example:

```
while(true){  
    System.out.println("This never ends.");  
}
```


Exercise - While the Sun is Out

18

```
int i = 0;           // initialization
while (i < 10) {      // loop condition
    System.out.println("I should be careful with the code that the AI wrote for me!");
    i = i + 1;        // increment
}
```

In Copenhagen, the summer days are long.

We love to sit on the grass outside the ITU building until the sun goes down at 21:00.

Declare a method named **isSunOut** that takes an **int** for the **time** and **returns** a **boolean**.

Use a **while loop** in the main function starting at 14:00.

The loop should keep running as long as the sun is out, checking the status and **incrementing** the time by 1 each round.

While Loops Quiz

- It is crucial to understand when a loop will exit and how the increment works inside the loop body.
- Let's take a look at the following code snippets and predict their exact output.
- An "infinite loop" is a loop never stops.

Exercise - While Loop Quiz 1

19

```
int i = 10;  
while (i > 0) {  
    System.out.println(i);  
    i = i - 1;  
}
```

Exercise - While Loop Quiz 2

20

```
int i = 1;
while (i <= 10) {
    System.out.println(i);
    i = i + 2;
}
```

Exercise - While Loop Quiz 3

21

```
int i = 1;
while (i < 100) {
    System.out.println(i);
    i = i * 2;
}
```

Exercise - While Loop Quiz 4

22

```
int i = 1;
while (i < 42) {
    System.out.println(i);
    i = i * i;
}
```


Exercise - While Loop Quiz 5

23

```
int i = 0;
while (i <= 15) {
    System.out.println(i);
    i = i + 3;
}
```

Exercise - While Loop Quiz 6

24

```
int i = 64;  
while (i >= 2) {  
    System.out.println(i);  
    i = i / 2;  
}
```

Exercise - While Loop

25

```
public static void main(String[] args) {  
    int i = 0; // initialization  
    while ( i < 10 ) { // loop condition  
        System.out.println("I should be careful with AI code!");  
        i = i + 1; // incrementation  
    }  
}
```

Cafe Analog has a mobile app where you can buy 5 tickets at once with a discount.

You are a heavy coffee drinker and want 50 tickets in one go — so you need a **loop** that buys a pack of **5 tickets, ten times**.

Using a **while loop**, print the running total after each pack: 5, 10, 15, ..., 50 (one number per line).

For Loops

- The **for loop** is a simpler way to do loops based on counting
- Any while loop can be a for loop, but for loops make the **counter** and **condition** clearer.

```
public static void main(String[] args) {  
    int i = 0; // initialization  
    while ( i < 10 ) { // loop condition  
        System.out.println("I should be careful with AI code!");  
        i = i + 1; // incrementation  
    }  
  
    for ( int i = 0; i < 10; i = i + 1; ) {  
        System.out.println("II should be careful with AI code!");  
    }  
}
```

i starts at 0 condition i < 10 i runs from 0 to 9 (10 times).

Exercise - For Loop

26

```
for ( int i = 0; i < 10; i = i + 1; ) {  
    System.out.println("II should be careful with AI code!");  
}
```

Cafe Analog has a mobile app where you can buy 5 tickets at once with a discount.

You are a heavy coffee drinker and want 50 tickets in one go — so you need a **loop** that buys a pack of **5 tickets, ten times**.

Rewrite the Cafe Analog tickets program. Same output (5 ... 50), but use a **for loop** this time.

Exercise - For Loop Quiz 1

27

```
for (int i = 10; i > 0; i = i - 2) {  
    System.out.println(i);  
}
```

Exercise - For Loop Quiz 2

28

```
for (int i = 1; i < 10; i = i + 3) {  
    System.out.println(i);  
}
```

Exercise - For Loop Quiz 3

29

```
for (int i = 1; i < 10; i = i * i) {  
    System.out.println(i);  
}
```


Exercise - For Loop Quiz 4

30

```
for (int i = 0; i <= 15; i = i + 3) {  
    System.out.println(i);  
}
```

Exercise - For Loop Quiz 5

31

```
for (int i = 1; i <= 10000; i = i * 10) {  
    System.out.println(i);  
}
```

Exercise - For Loop Quiz 6

32

```
for (int i = 64; i >= 2; i = i / 2) {  
    System.out.println(i);  
}
```


For Loop vs. While Loop

- **for loops** are more common than **while loops**
- They're more **compact**
- They're **easier** to read once you're used to it
- They are harder to mess up (it's really easy to forget the increment when making a while loop)
- The "**counter**" variables often have short names:
 - i, j, k
 - x, y, z

Recap: Anatomy of Java Code

- **Statement:** A complete unit of execution (usually ending with a semicolon ;).
- **Expression:** A code snippet that evaluates to a single value.
- **Literal Value:** A fixed value written directly in the source code.
- **Variable:** A named storage location of a specific type that holds a value.

Variable Literal Value

`int x = 2;`

`System.out.println((4 + (x * x) + (4 - x)) / x);`

Expressions

Statement

Expressions

- **Constant (42)**
- **Variable (x)**
- **Comparison** between expressions
 - $\text{Exp} == \text{Exp}$
 - $\text{Exp} != \text{Exp}$
 - $\text{Exp} > \text{Exp}$
 - $\text{Exp} < \text{Exp}$
 - $\text{Exp} >= \text{Exp}$
 - $\text{Exp} <= \text{Exp}$
- **Operation** between expressions
 - $\text{Exp} + \text{Exp}$
 - $\text{Exp} - \text{Exp}$
 - $\text{Exp} * \text{Exp}$
 - Exp / Exp
 - $\text{Exp} \% \text{Exp}$
- **Logical Operators**
 - $\text{Exp} \&\& \text{Exp}$
 - $! \text{Exp}$
 - $\text{Exp} || \text{Exp}$

Statements

- **Assignment**
 - Variable = Expressions
 - Variable = Constant
- **Conditional**
 - `if(expression) { statement }`
 - `if(expression) { statement } else { statement }`
- **Loops**
 - `while(expression) { statement }`
 - `for(initialization, expression, iteration) { statement }`

Nested for Loops

Sometimes you need two counters.

A nested for reads like the shape of the data:

```
public class Main {  
    public static void main(String[] args) {  
        for (int y = 0; y < 3; y++) {  
            for (int x = 0; x < 3; x++) {  
                System.out.print("(" + x + ", " + y + ") ");  
            }  
            System.out.println();  
        }  
    }  
}
```

(0, 0)	(1, 0)	(2, 0)
(0, 1)	(1, 1)	(2, 1)
(0, 2)	(1, 2)	(2, 2)

Exercise - Using Nested for loops

33

```
public class Main {  
    public static void main(String[] args) {  
        for (int y = 0; y < 3; y++) {  
            for (int x = 0; x < 3; x++) {  
                System.out.print("(" + x + ", " + y + ") ");  
            }  
            System.out.println();  
        }  
    }  
}
```

You are in the ITU fitness room. Today's plan: 4 sets, 12 reps each.

Use a **nested for** to log every rep — outer loop = set (1...4), inner loop = rep (1...12).

The program should print :

Set 1 Rep 1

Set 1 Rep 2

Set 1 Rep 3

...

Set 4 Rep 12

Exercise - More Loops Practice

34

```
public static void main(String[] args) {  
  
    int i = 0; // initialization  
    while ( i < 10 ) { // loop condition  
        System.out.println("I should be careful with AI code!");  
        i = i + 1; // incrementation  
    }  
  
    for ( int i = 0; i < 10; i = i + 1; ) {  
        System.out.println("II should be careful with AI code!");  
    }  
}
```

Cafe Analog wants fewer disposable cups. Every time you buy a drink and bring your own cup, you get a stamp on your card — the 10th stamp is a free cup, and you start a fresh card right after.

Simulate 24 drinks bought (one per loop iteration) with a **for or while loop**: stamps **starts at 0**, and each drink adds one stamp.

- While **stamps != 10**: print "Brought my own cup, got a stamp! ({stamps}/10)".
- The moment **stamps == 10**: print "Free cup! Here's a new stamp card." instead, then reset stamps back to 0.



Agenda

- Recap
- Conditional Statements
 - If Statements
 - If-Else Statements
 - Nested if(-else) Statements
 - If-Else-If Ladder
- Booleans, Logical and Comparison Operators
- Returning Functions
- Iterations / Loops
- **Interactive Programs**

Random

Think of Java's **utilities** as a built-in toolbox. You can use these tools by **importing** them at the top of your program.

java.util.Random is one of them, which generates pseudo-random numbers.

For example, calling `nextInt(4)` produces 0, 1, 2, or 3 (each 25% likely).

```
import java.util.Random;

public class Main {
    public static void main(String[] args) {
        Random rng = new Random();
        int roll = rng.nextInt(4);
        if (roll == 0) {
            System.out.println("Zero");
        }
    }
}
```

Exercise - Using Random in a Loop

35

Friday at Scrollbar means one thing: beer pong! Let's set up the rack and simulate a full game.

Part 1: Use a **nested for loop** to print a 4-row triangle cup rack.

(Hint: Think about how the row number relates to the number of cups in that row).

Expected output for this part: Row 1: O Row 2: O O Row 3: O O O Row 4: O O O O

Part 2: Simulate throws using a **while loop** until all 10 cups are cleared.

You will need to keep track of the **remaining cups** and the **total number of throws**.

Game Rules:

- For each throw, use **java.util.Random** to generate a random number between 0 and 1 (inclusive).
- Treat a roll of 0 as a hit (a 50% chance). A hit removes one cup.
- A roll of 1 is a miss.
- Print the result of every throw. When the rack is empty, print a final game over message.

Expected output format for Part 2:

Throw 1: MISS! 10 cups left. Throw 2: SPLASH! 9 cups left. Throw 3: MISS! 9 cups left. ... Throw 17: SPLASH! 0 cups left. GAME OVER — the rack is empty. Chug up!

Scanner (User Input)

Just like Random, Scanner is another essential tool you can grab from Java's built-in toolbox.

Scanner allows your program to **pause** and **listen** to the **user's keyboard input**.

By importing **java.util.Scanner** at the top of your program, you can ask questions, read the answers, and make your code truly interactive!

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Hello there!");
        System.out.print("What is your name? ");

        String userName = scanner.nextLine();

        System.out.println("Nice to meet you, " + userName + "!");

        scanner.close();
    }
}
```

Bonus Exercise - Using Scanner in VS Code



```
import java.util.*;
class Guess {
public static void main(String[] args) {
    Random random = new Random();
    Scanner scanner = new Scanner(System.in);
    int secret = random.nextInt(100);
    int tries = 0;
    int guess = -1;
    // As long as the correct number has not been guessed{
        // output make a guess
        guess = scanner.nextInt(); // user inputs guess
        // update tries
        // if guess is less than secret: print 'too low'
        // if guess is higher than secret: print 'too high'
    // }
    // print a congratulations message
    // print 'you used ... guesses'
    }
}
```